

UG PROJECT 2025-26
EC22711 - Project Work Phase - II

SoCForge: Automated SoC Builder

External Supervisor

Dr Arun Janarthanan

Technology Director and Engineering

Practice, Edtech business, HCLtech

Internal Supervisor

Dr T. J. Jeyaprabha

Associate Professor

Department of ECE

Members of students: (A20)

Balasaraswathy B(Reg. No.: 2127220701022)

Haresh Krishna G S(Reg. No.: 2127220701043)

Harinee V T(Reg. No.: 2127220701047)

Project Title & Domain

Project Title & Domain Title

- **TITLE** : SoCForge: Automated SoC Builder
- **DOMAIN** : Very Large Scale Integration (VLSI)



Problem Statement

- Traditional SoC design flows rely heavily on manual integration connecting IPs, resolving interface mismatches, and writing top-level RTL resulting in extensive design time, higher error rates, and poor scalability with growing system complexity. This manual approach also complicates verification and makes architectural changes slow and error-prone.
- Modern System-on-Chip (SoC) designs integrate numerous IP blocks such as processors, memory controllers, interconnects, and hardware accelerators, demanding high performance, low power, and minimal area.
- **Need:** To develop an automated SoC generation framework capable of reading a high-level YAML configuration that describes the system architecture, validating IP compatibility, generating complete synthesizable RTL. Such automation aims to reduce manual integration effort, eliminate human-induced design errors, and accelerate the SoC design cycle.



Objectives of the Project

- **Goal:** Design and implement an automated SoC generation tool that creates complete top-level RTL directly from a YAML configuration file.
- **SoC Integration Objective:** Build a working microcontroller-class SoC by integrating a RISC-V CPU(PicoRV32), memory blocks, and peripherals using automatically generated interconnect and address decoding logic.
- **Automation Objective:** Replace manual RTL glue logic (address mapping, bus wiring, and peripheral connections) with rule-based automatic generation.
- **Verification:** Perform RTL functional verification with Icarus Verilog and inspect waveforms using GTKWave.
- **Outcome:** Automated generation of SoC top-level RTL from YAML input.



Proposed Solution/Methodology

Overview

- The proposed system aims to automate the SoC design flow, enabling generation of a top-level RTL implementation from a user-defined YAML configuration file.
- This approach eliminates repetitive manual integration, reduces design complexity, and ensures consistent, reproducible design flows.
- This tool consumes structured YAML specifications describing CPUs, memories, peripherals, and bus policies, automatically derives address maps, constructs connectivity models, and emits structural Verilog code.
- The framework provides flexibility for defining processor cores, memory blocks, peripherals, and accelerators in a modular, easily configurable manner.

Key Objectives

- Automation of top-level RTL integration
- Modular and stage-separated architecture
- Fabric-agnostic front-end design
- Support for multiple interconnect fabrics
- Continuous verification at every stage



Outcome of Phase I

Phase 1: Manual SoC Implementation

Objective: To develop a **reference** SoC manually to establish a baseline RTL-to-GDSII flow and validate functional and physical integration.

Description:

A PicoRV32-based SoC is manually constructed, integrating the following key components:

- PicoRV32 – RISC-V CPU core
- SPI Flash interface – for program storage
- SRAM – for runtime data and stack memory
- UART – for serial communication
- 64-point FFT Accelerator – boosts signal processing for real-time performance

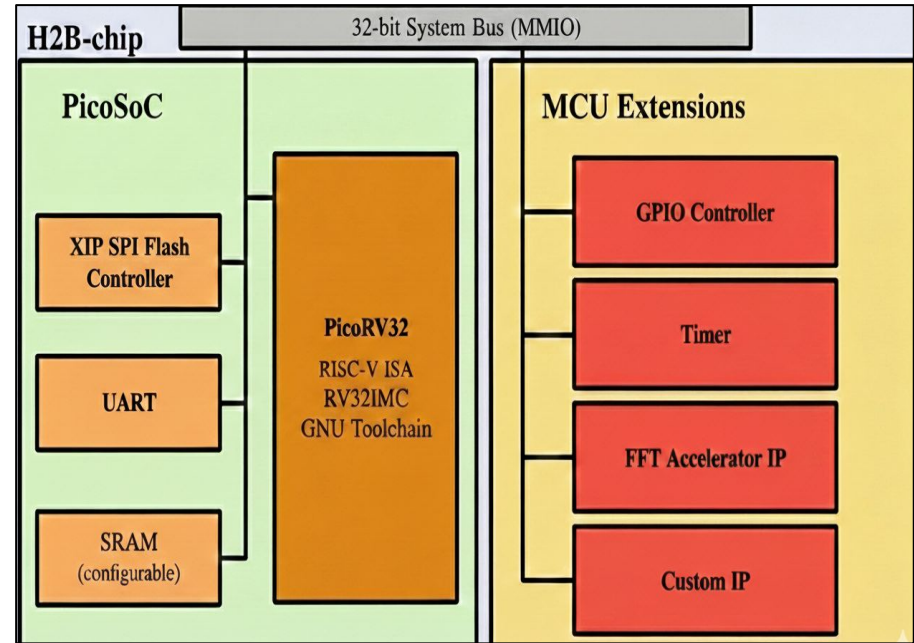
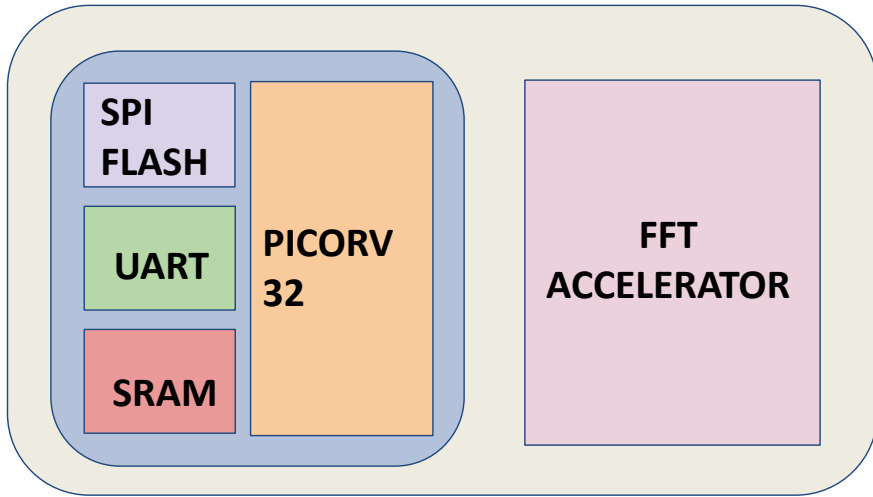
Outcome:

Established a reference flow and design template to serve as the foundation for automation in Phase 2.

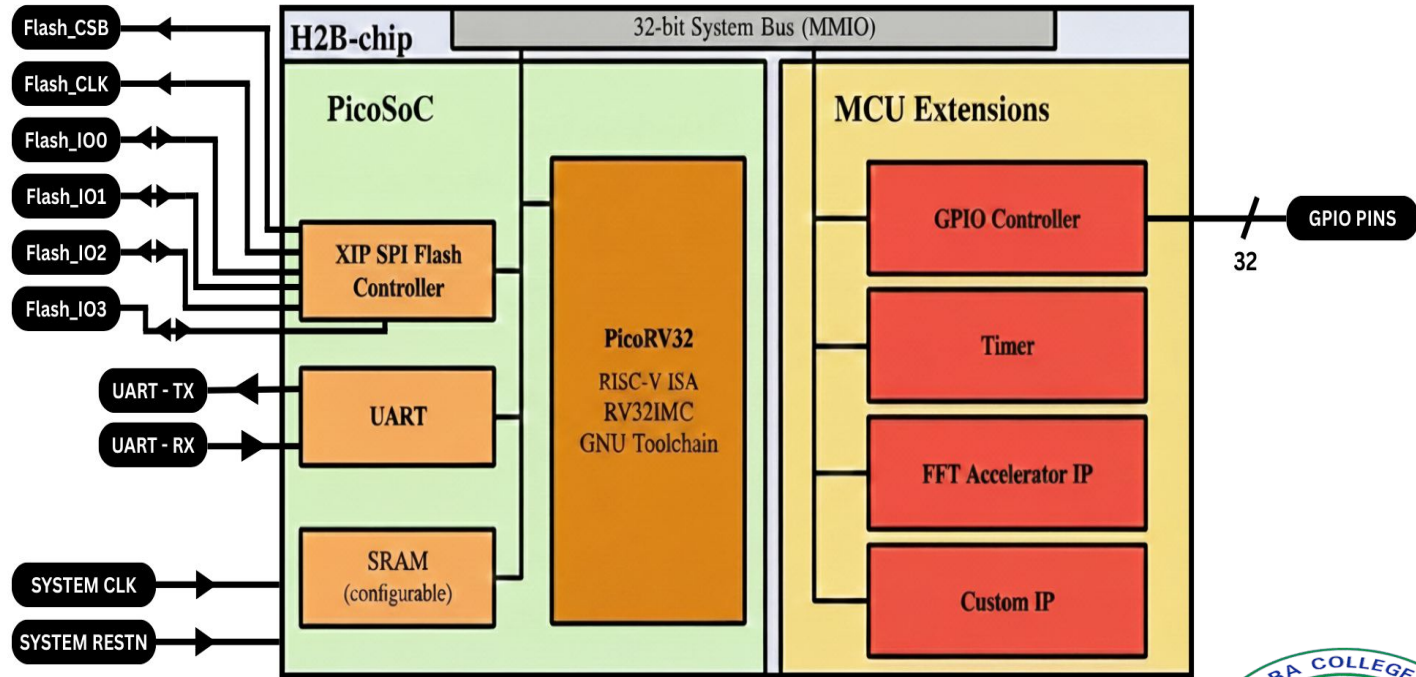
Flow Execution:

- RTL Design: Manual Verilog integration and top-level connectivity definition
- Synthesis: Performed using Yosys for logic optimization and netlist generation
- Floor planning, Placement, and Routing: Implemented using OpenROAD toolchain
- Sign-off: Includes timing, DRC, and LVS verification
- Output: Fully verified GDSII layout ready for fabrication

Pico SoC Block Diagram: Phase 1 & Phase 2 Comparison



Pico SoC Block Diagram



Proposed Solution/Methodology

Phase 2: YAML-Based Automation Framework

Objective:

To build an **Automated SoC Builder** that can generate top level RTL **directly from a YAML specification**.

Advantages:

- Eliminates manual design overhead
- Reduces integration time and human error
- Enables rapid exploration of multiple SoC architectures
- Promotes reuse of verified IP components

Outcome

A YAML-driven automation framework that converts user-defined SoC architecture specifications into synthesizable top-level RTL through a streamlined front-end design pipeline.

System Workflow:

YAML Input Parsing:

- Reads user-defined architecture (CPU configuration, Memories and peripherals, Address regions, Interrupt assignments , Interconnect fabric selection)
- Extracts parameters like address map and bus interfaces.
- Calculates data width to avoid address overlapping.

Validation & Compatibility Checks:

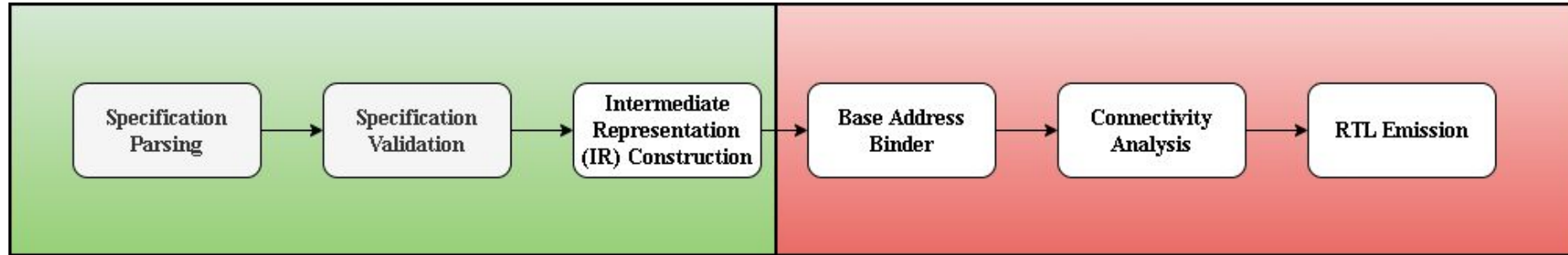
- Each peripheral must define an address aperture and alignment
- IRQ numbers must be within the supported range
- Selected interconnect fabric must be supported
- Address overlapping has to be avoided

Automated RTL Generation:

- Generates top-level Verilog interconnects and memory maps
- Creates wrapper modules for IP integration



Project Workflow



Project Workflow

Stage 1: Specification Parser

The specification parser is responsible solely for reading YAML input files and converting them into in-memory Python dictionaries. It performs no validation, defaulting, or interpretation of the data.

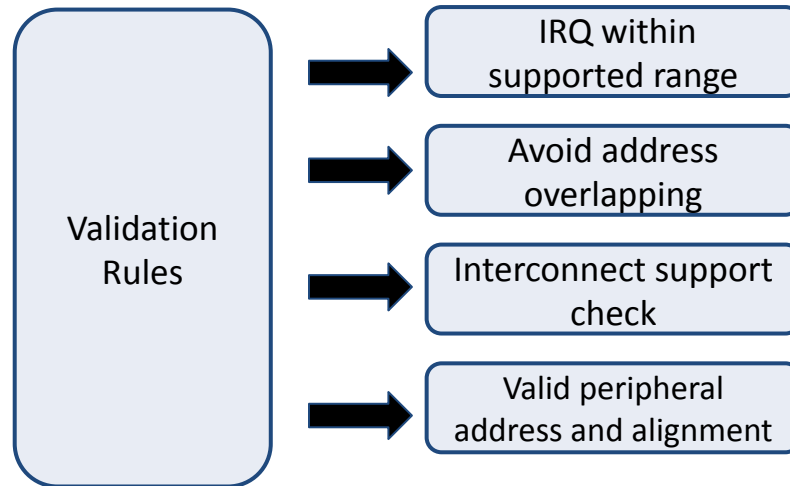


The parser exists to isolate file I/O concerns from architectural logic, allowing the specification format to evolve independently.

Project Workflow

Stage 2: Specification Validator

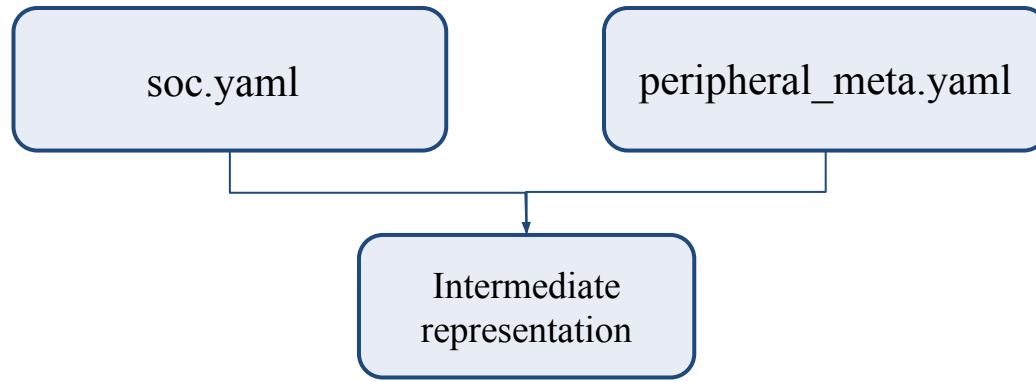
The validator enforces correctness of the parsed specification before any architectural decisions are made. It checks for missing required fields, invalid values, duplicate identifiers, and illegal combinations of parameters.



Project Workflow

Stage 3: Intermediate Representation Builder

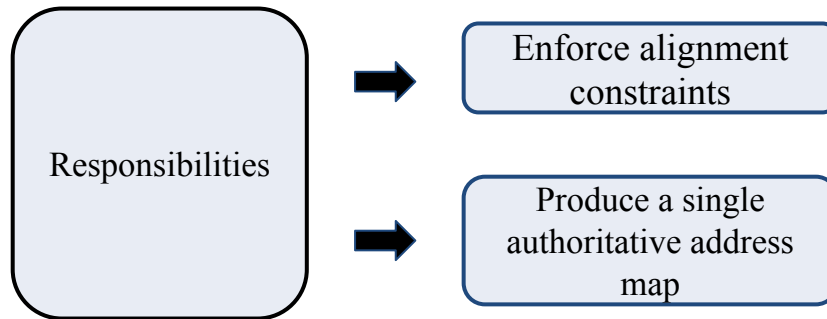
- The IR builder converts the validated specifications into typed internal objects representing the CPU, memory regions, peripherals, buses, and interrupts.
- Instance-specific information such as base addresses, sizes, IRQs, and parameters is taken from **soc.yaml**, while peripheral interface semantics including supported buses, signal roles, adapters, register definitions, and top-level ports are imported from **peripheral_meta.yaml**.



Project Workflow

Stage 4: Base Address Binder

The Base address binder assigns base addresses to memories and peripherals based on declared size and alignment constraints. The allocation is deterministic and architecture-independent.



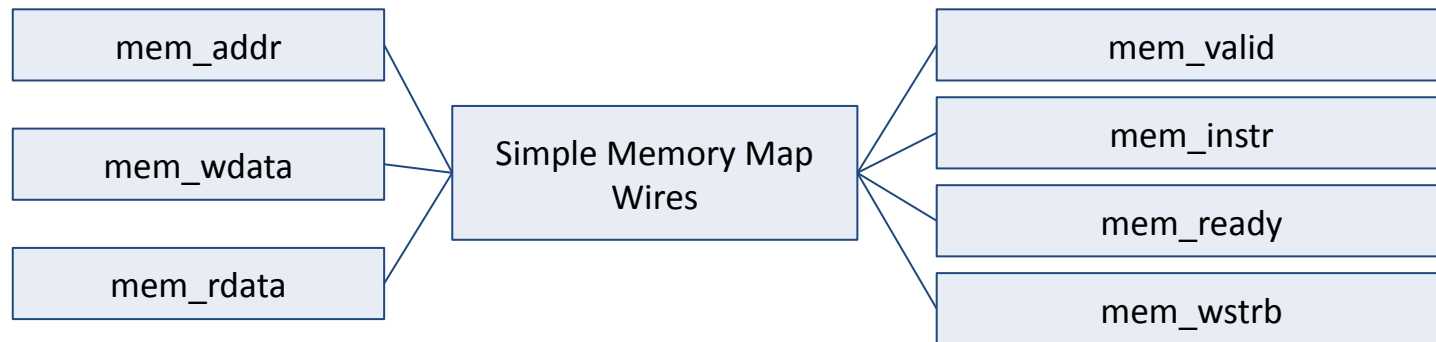
Project Workflow

Stage 5: Connectivity Analysis

The connectivity builder determines how components communicate over the selected interconnect fabric. This is the stage where architectural divergence occurs.

Simple Memory-Mapped Connectivity

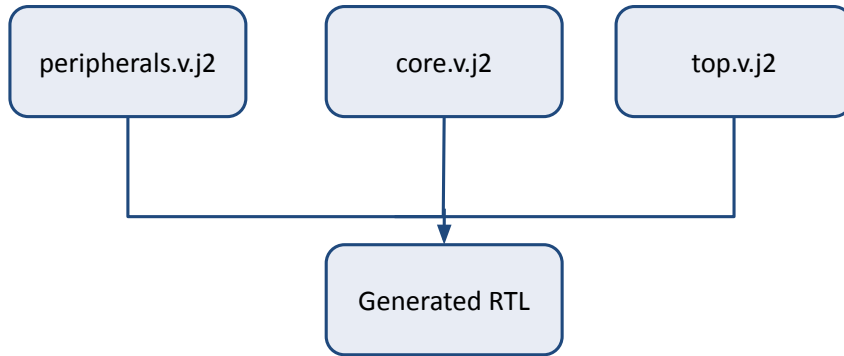
The Simple MM fabric assumes a single master and multiple slaves with at most one outstanding transaction. Connectivity consists of combinational address decoding, priority-based slave selection, and simple response multiplexing.



Project Workflow

Stage 6: RTL Emitter

- The RTL emitter translates connectivity models into synthesizable Verilog using parameterized templates. It generates only structural RTL and contains no architectural decision-making logic.
- RTL is generated using Jinja-based parameterized templates, where template files with the extension .vj2 represent Verilog (.v) code with Jinja placeholders that are resolved during generation.



```
|-- templates/
| |-- top_level.v.j2
| |-- core/
| | |-- picorv32.v.j2
| | |-- picorv32_params.v.j2
| |-- peripherals/
| | |-- fft_accel/
| | | |-- iface_simple_mem.v.j2
| | |-- simpleuart/
| | | |-- iface_simple_mem.v.j2
| | |-- spimemio/
| | | |-- iface_simple_mem.v.j2
```

Jinja template that generates the top-level SoC RTL

Jinja template for core instantiation

Jinja template for FFT instantiation

Jinja template for UART instantiation

Jinja template for SPI instantiation



Sample .v.j2 for AND Gate

```
module {{ module_name }} (  
    input logic [{{ width-1 }}:0] a,  
    input logic [{{ width-1 }}:0] b,  
    output logic [{{ width-1 }}:0] y  
);  
  
{% for i in range(width) %}  
assign y[{{ i }}] = a[{{ i }}] & b[{{ i }}];  
{% endfor %}  
  
endmodule
```

- `{{ }}` is used to output variables or expressions in a Jinja2 template.
- Allows dynamic content generation, replacing placeholders with actual values when rendering.
- `{% %}` is used for **control statements** in Jinja2 templates.

TOOL DIRECTORY STRUCTURE

```
Automation/
|-- Firmware/
|   |--firmware.c
|   |--firmware.hex
|   |--firmware.elf
|   |--Start.s
|   |--sections.lds
|-- IP/
|   |--CORE/
|   |   |--picorv32.v
|   |   |--FFT/
|   |   |   |--Butterfly.v
|   |   |   |--DelayBuffer.v
|   |   |   |--FFT64.v
|   |   |   |--fft_accel.v
|   |   |   |--Multiply.v
|   |   |   |--SdfUnit.v
|   |   |   |--TB64.v
|   |   |   |--Twiddle64.v
|   |   |--SPI/
|   |   |   |--spimemio.v
|   |   |   |--spiflash.v
|   |   |   |--spiflash_tb.v
|   |   |--UART/
|   |   |   |--simpleuart.v
|-- ir/
|   |-- builder.py
|   |-- models.py
|   |-- peripheral_meta.yaml
|-- templates/
|   |-- top_level.v.j2
|   |-- core/
|   |   |-- picorv32.v.j2
|   |   |-- picorv32_params.v.j2
|   |-- peripherals/
|   |   |-- fft_accel/
|   |   |   |-- iface_simple_mem.v.j2
|   |   |-- simpleuart/
|   |   |   |-- iface_simple_mem.v.j2
|   |   |-- spimemio/
|   |   |   |-- iface_simple_mem.v.j2
|-- schema/
|   |-- soc.schema.json
|-- specs/
|   |-- soc.yaml
|   |-- soc_invalid_schema.yaml
|   |-- soc_irq.yaml
|   |-- soc_overlap.yaml
|-- tests/
|-- validate/
|   |-- schema_validate.py
|   |-- semantic_validate.py
|-- main.py
```

STAGE 1

SPECIFICATION PARSING



STAGE 1: YAML FILE TEMPLATE AND SPECIFICATIONS

Template

```
cpu:
  type:
  parameters:
    STACKADDR:
    PROGADDR_RESET:
    PROGADDR_IRQ:
    BARREL_SHIFTER:
    ENABLE_MUL:
    ENABLE_DIV:
    ENABLE_FAST_MUL:
    ENABLE_COMPRESSED:
    ENABLE_COUNTERS:
    ENABLE_IRQ:
    ENABLE_IRQ_QREGS:
```

Example

```
cpu:
  type: picorv32
  parameters:
    STACKADDR: 0x00000400
    PROGADDR_RESET: 0x00100000
    PROGADDR_IRQ: 0x00000000
    BARREL_SHIFTER: true
    ENABLE_MUL: true
    ENABLE_DIV: true
    ENABLE_FAST_MUL: false
    ENABLE_COMPRESSED: true
    ENABLE_COUNTERS: true
    ENABLE_IRQ: true
    ENABLE_IRQ_QREGS: false
```

STAGE 1: YAML FILE TEMPLATE AND SPECIFICATIONS

Template

```
peripherals:  
  - name:  
    type:  
    base_addr:  
    size:  
    bus:  
    parameters:
```

Example

```
peripherals:  
  - name: spimemio  
    type: spimemio  
    base_addr: 0x02000000  
    size: 0x00000003  
    bus: simple_mem  
    parameters: {}  
  
  - name: uart0  
    type: simpleuart  
    base_addr: 0x02000004  
    size: 0x00000004  
    bus: simple_mem  
    irq: uart_irq  
    parameters:  
      baudrate: 115200
```

STAGE 1: YAML FILE TEMPLATE AND SPECIFICATIONS

Template

```
bus:
  type:
  addr_width:
  data_width:
memory:
  - name:
    type:
    base_addr:
    size:
    parameters:
      word_width:
```

Example

```
bus:
  type: simple_mem
  addr_width: 32
  data_width: 32
memory:
  - name: sram
    type: onchip_ram
    base_addr: 0x00000000
    size: 0x00000400
    parameters:
      word_width: 32
```

STAGE 1: YAML FILE TEMPLATE AND SPECIFICATIONS

Template

```
interrupts:  
  width:  
  assignments:  
    uart_irq:  
    irq_5:  
    irq_6:  
    irq_7:  
    fft_irq:
```

Example

```
interrupts:  
  width: 32  
  assignments:  
    uart_irq: 4  
    irq_5: 5  
    irq_6: 6  
    irq_7: 7
```

STAGE 1: OUTPUT

```
yaml_dict
{'comments': 'This is a YAML specification for a PicoSoC system-on-chip (SoC) configuration.\nIt def...ligned properly.\nAuthor: Balasaraswathy B\n', 'soc': {'name': 'picosoc', 'clock': {...}, 'reset': {...}, 'cpu': {'type': 'picorv32', 'parameters': {...}, 'bus': {'type': 'simple_mem', 'addr_width': 32, 'data_width': 32}, 'memory': [...], 'peripherals': [...], 'interrupts': {'width': 32, 'assignments': {...}}}
> special variables
> function variables
  'comments' = 'This is a YAML specification for a PicoSoC system-on-chip (SoC) configuration.\nIt defines the CPU, bus, memory, peripherals, and interrupt assign
> 'soc' = {'name': 'picosoc', 'clock': {'name': 'clk', 'frequency_hz': 50000000}, 'reset': {'name': 'resetn', 'active_low': True}}
> 'cpu' = {'type': 'picorv32', 'parameters': {'STACKADDR': 1024, 'PROGADDR_RESET': 1048576, 'PROGADDR_IRQ': 0, 'BARREL_SHIFTER': True, 'ENABLE_MUL': True, 'ENABLE
> 'bus' = {'type': 'simple_mem', 'addr_width': 32, 'data_width': 32}
> 'memory' = [{'name': 'sram', 'type': 'onchip_ram', 'base_addr': 0, 'size': 1024, 'parameters': {...}}]
> 'peripherals' = [{'name': 'spimemio', 'type': 'spimemio', 'base_addr': 33554432, 'size': 3, 'bus': 'simple_mem', 'parameters': {}}, {'name': 'uart0', 'type': 's
> special variables
> function variables
> 0 = {'name': 'spimemio', 'type': 'spimemio', 'base_addr': 33554432, 'size': 3, 'bus': 'simple_mem', 'parameters': {}}
> 1 = {'name': 'uart0', 'type': 'simpleuart', 'base_addr': 33554436, 'size': 4, 'bus': 'simple_mem', 'irq': 'uart_irq', 'parameters': {'baudrate': 115200}}
> 2 = {'name': 'fft0', 'type': 'fft_accel', 'base_addr': 50331648, 'size': 512, 'bus': 'simple_mem', 'irq': 'fft_irq', 'parameters': {}}
  len() = 3
> 'interrupts' = {'width': 32, 'assignments': {'uart_irq': 4, 'irq_5': 5, 'irq_6': 6, 'irq_7': 7, 'fft_irq': 8}}
> special variables
> function variables
  'width' = 32
> 'assignments' = {'uart_irq': 4, 'irq_5': 5, 'irq_6': 6, 'irq_7': 7, 'fft_irq': 8}
  len() = 2
  len() = 7
```

Converted python dictionaries from Yaml

STAGE 2

SPECIFICATION VALIDATION



STAGE 2: UNDEFINED IRQ ERROR

INPUT

```
peripherals:
- name: uart0
  type: simpleuart
  base_addr: 0x02000000
  size: 0x8
  irq: unknown_irq      # IRQ
  parameters:
    baudrate: 115200

interrupts:
  width: 32
  assignments:
    uart_irq: 4
```

OUTPUT

```
File "C:\Users\balas\OneDrive\Desktop\r22sdf\Automation\validate_semantic.py", line 10, in validate
    raise ValueError(f"Peripheral {p.name} uses undefined IRQ {p.irq}")
ValueError: Peripheral uart0 uses undefined IRQ unknown_irq
PS C:\Users\balas\OneDrive\Desktop\r22sdf\Automation>
```

In this stage, the IRQ names are validated and as shown in the input, the irq name “*unknown_irq*” doesn’t match with the expected name which is “*uart_irq*” and hence the error is thrown in the output terminal.

STAGE 2: ADDRESS OVERLAP ERROR

INPUT

```
memory:
- name: sram
  type: onchip_ram
  base_addr: 0x00000000
  size: 0x00000400
  parameters:
    word_width: 32

peripherals:
- name: spiflash
  type: spimemio
  base_addr: 0x00000000 # Overlaps with sram
  size: 0x00001000
  parameters: {}

interrupts:
  width: 32
  assignments: {}
```

OUTPUT

```
ValueError: Address overlap detected: spiflash
PS C:\Users\balas\OneDrive\Desktop\r22sdf\Automation>
```

Here, the *SRAM* and *SpiFlash* are given the same base address; hence, the error is thrown in the terminal.

STAGE 2: MISSING PROPERTY ERROR

INPUT

```
5 soc:
6   name: test_soc
7   reset:
8     name: resetn
9     active_low: true
10
11 cpu:
12   type: picorv32
13   parameters:|
14     STACKADDR: 0x00000400
15     ENABLE_MUL: true
16
17 bus:
18   type: simple_mem
19   addr_width: 32
20   data_width: 32
21
22 memory: []
23 peripherals: []
24 interrupts:
25   width: 32
26   assignments: {}
27
```

OUTPUT

```
jsonschema.exceptions.ValidationError: 'clock' is a required property

Failed validating 'required' in schema['properties']['soc']:
  {'type': 'object',
   'required': ['name', 'clock', 'reset'],
   'properties': {'name': {'type': 'string'},
                  'clock': {'type': 'object',
                           'required': ['name', 'frequency_hz'],
                           'properties': {'name': {'type': 'string'},
                                          'frequency_hz': {'type': 'integer'}}},
                  'reset': {'type': 'object',
                           'required': ['name', 'active_low'],
                           'properties': {'name': {'type': 'string'},
                                          'active_low': {'type': 'boolean'}}}}}

On instance['soc']:
  {'name': 'broken_soc', 'reset': {'name': 'resetn', 'active_low': True}}
PS C:\Users\balas\OneDrive\Desktop\r22sdf\Automation>
```

Here, the required parameter '**clock**' is missing; hence, the error is thrown in terminal.

STAGE 3

INTERMEDIATE REPRESENTATION



Peripheral_meta.yaml

```
7 simpleuart:
8   supported_buses:
9     simple_mem:
10       adapter: simple_mem_adapter
11       roles:
12         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
13   top_ports:
14     ser_tx: { dir: output }
15     ser_rx: { dir: input }
16   parameters:
17     registers:
18       reg_div:
19         wait: false
20       reg_dat:
21         wait: true
22 spimemio:
23   supported_buses:
24     simple_mem:
25       adapter: simple_mem_adapter
26       roles:
27         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
28         resp: { ready: "spimem_ready", rdata: "spimem_rdata" }
29   parameters:
30     registers:
31       cfgreg :
32   top_ports:
33     flash_csb: { dir: output }
34     flash_clk: { dir: output }
35     flash_io0_oe: { dir: output }
36     flash_io1_oe: { dir: output }
37     flash_io2_oe: { dir: output }
```

```
Automation > ir > ! peripheral_meta.yaml
22 spimemio:
23   supported_buses:
24     simple_mem:
25       adapter: simple_mem_adapter
26       roles:
27         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
28         resp: { ready: "spimem_ready", rdata: "spimem_rdata" }
29   parameters:
30     registers:
31       cfgreg :
32   top_ports:
33     flash_csb: { dir: output }
34     flash_clk: { dir: output }
35     flash_io0_oe: { dir: output }
36     flash_io1_oe: { dir: output }
37     flash_io2_oe: { dir: output }
38     flash_io3_oe: { dir: output }
39     flash_io0_do: { dir: output }
40     flash_io1_do: { dir: output }
41     flash_io2_do: { dir: output }
42     flash_io3_do: { dir: output }
43     flash_io0_di: { dir: input }
44     flash_io1_di: { dir: input }
45     flash_io2_di: { dir: input }
46     flash_io3_di: { dir: input }
47
48 fft_accel:
49   supported_buses:
50     simple_mem:
51       adapter: simple_mem_adapter
52       roles:
53         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
54         resp: { ready: "fft_ready", rdata: "fft_rdata" }
55   parameters: {}
56
57
```

For a new peripheral it's metadata has to be added here

Peripheral_meta.yaml

```
1 comments: |
2   File name: peripheral_meta.yaml
3   Description: This file defines the metadata for various peripherals used in the SoC.
4   Each peripheral specifies its supported buses, top-level ports, and parameters.
5   Modify or add peripherals as needed for your SoC design.
6   Author: Balasaraswathy B Hareesh Krishna G S
7
8 simpleuart:
9   rtl:
10     - /home/hkremote/Desktop/tempfile/IP/UART/simpleuart.v
11   supported_buses:
12     simple_mem:
13       adapter: simple_mem_adapter
14       roles:
15         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
16   top_ports:
17     ser_tx: { dir: output }
18     ser_rx: { dir: input }
19   parameters:
20     registers:
21       reg_div:
22         wait: false
23       reg_dat:
24         wait: true
25
26 spimemio:
27   rtl:
28     - /home/hkremote/Desktop/tempfile/IP/SPI/spimemio.v
29   supported_buses:
30     simple_mem:
31       adapter: simple_mem_adapter
32       roles:
33         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
34         resp: { ready: "spimem_ready", rdata: "spimem_rdata" }
35   parameters:
36     registers:
37       cfgreg:
38   top_ports:
39     flash_csb: { dir: output }
40     flash_clk: { dir: output }
41     flash_io0_oe: { dir: output }
42     flash_io1_oe: { dir: output }
43     flash_io2_oe: { dir: output }
44     flash_io3_oe: { dir: output }
45     flash_io0_do: { dir: output }
46     flash_io1_do: { dir: output }
47     flash_io2_do: { dir: output }
48     flash_io3_do: { dir: output }
49     flash_io0_di: { dir: input }
50     flash_io1_di: { dir: input }
51     flash_io2_di: { dir: input }
52     flash_io3_di: { dir: input }
```

```
53
54 fft_accel:
55   rtl:
56     - /home/hkremote/Desktop/tempfile/IP/FFT/fft_accel.v
57   supported_buses:
58     simple_mem:
59       adapter: simple_mem_adapter
60       roles:
61         req: { valid: "mem_valid", addr: "mem_addr", wdata: "mem_wdata", wstrb: "mem_wstrb" }
62         resp: { ready: "fft_ready", rdata: "fft_rdata" }
63   parameters: {}
64
```

For a new peripheral it's metadata has to be added here

STAGE 3 OUTPUT

```
1
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
Filter (e.g. text, lexclude, \esca... Python Debugger: Curr...
soc_ir
SoCIR(name='picosoc', clock=ClockIR(name='clk', frequency_hz=50000000), reset=ResetIR(name='resetn', active_low=True), cpu=CPUConfigIR(type='picorv32', parameters={'STACKADDR': 1024, 'PROGADDR_RESET': 1048576, 'PROGADDR_IRQ': 0, 'BARREL_SHIFTER': True, 'ENABLE_MUL': True, 'ENABLE_DIV': True, 'ENABLE_FAST_MUL': False, 'ENABLE_COMPRESSED': True, 'ENABLE_COUNTERS': True, 'ENABLE_IRQ': True, 'ENABLE_IRQ_QREGS': False, 'MEM_WORDS': 256}, bus=BusIR(type='simple_mem', addr_width=32, data_width=32)), memory=[MemoryRegionIR(name='sram', type='onchip_ram', region=AddressRegionIR(base_addr=0, size=1024), parameters={'word_width': 32})], peripherals=[<ir.models.PeripheralIR object at 0x000001D269C6C770>, <ir.models.PeripheralIR object at 0x000001D269C6E3F0>, <ir.models.PeripheralIR object at 0x000001D269C6F050>], interrupts=InterruptMapIR(width=32, assignments={'uart_irq': 4, 'irq_5': 5, 'irq_6': 6, 'irq_7': 7, 'fft_irq': 8}))
> special variables
> clock = ClockIR(name='clk', frequency_hz=50000000)
> cpu = CPUConfigIR(type='picorv32', parameters={'STACKADDR': 1024, 'PROGADDR_RESET': 1048576, 'PROGADDR_IRQ': 0, 'BARREL_SHIFTER': True, 'ENABLE_MUL': True, 'ENABLE_DIV': True, 'ENABLE_FAST_MUL': False, 'ENABLE_COMPRESSED': True, 'ENABLE_COUNTERS': True, 'ENABLE_IRQ': True, 'ENABLE_IRQ_QREGS': False, 'MEM_WORDS': 256}, bus=BusIR(type='simple_mem', addr_width=32, data_width=32)), memory=[MemoryRegionIR(name='sram', type='onchip_ram', region=AddressRegionIR(base_addr=0, size=1024), parameters={'word_width': 32})], peripherals=[<ir.models.PeripheralIR object at 0x000001D269C6C770>, <ir.models.PeripheralIR object at 0x000001D269C6E3F0>, <ir.models.PeripheralIR object at 0x000001D269C6F050>], interrupts=InterruptMapIR(width=32, assignments={'uart_irq': 4, 'irq_5': 5, 'irq_6': 6, 'irq_7': 7, 'fft_irq': 8}))
> interrupts = InterruptMapIR(width=32, assignments={'uart_irq': 4, 'irq_5': 5, 'irq_6': 6, 'irq_7': 7, 'fft_irq': 8}))
> memory = [MemoryRegionIR(name='sram', type='onchip_ram', region=AddressRegionIR(base_addr=0, size=1024), parameters={'word_width': 32})]
name = 'picosoc'
> peripherals = [<ir.models.PeripheralIR object at 0x000001D269C6C770>, <ir.models.PeripheralIR object at 0x000001D269C6E3F0>, <ir.models.PeripheralIR object at 0x000001D269C6F050>]
> special variables
> function variables
> 0 = <ir.models.PeripheralIR object at 0x000001D269C6C770>
> special variables
> bus_bindings = {'simple_mem': PeripheralBusBindingIR(bus_type='simple_mem', adapter='simple_mem_adapter', roles=Bus... 'spimem_ready', 'rdata': 'spimem_rdata')}}
bus_type = 'simple_mem'
irq = None
name = 'spimemio'
> parameters = {'registers': {'cfgreg': None}}
> region = AddressRegionIR(base_addr=33554432, size=3)
> top_ports = {'flash_csb': {'dir': 'output'}, 'flash_clk': {'dir': 'output'}, 'flash_io0_oe': {'dir': 'output'}, 'flash_io1_oe': {'dir': 'output'}, 'flash_io2_oe': {'dir': 'output'}, 'flash_io3_oe': {'dir': 'output'}}
type = 'spimemio'
> 1 = <ir.models.PeripheralIR object at 0x000001D269C6E3F0>
> special variables
> bus_bindings = {'simple_mem': PeripheralBusBindingIR(bus_type='simple_mem', adapter='simple_mem_adapter', roles=Bus... 'm_wdata', 'wstrb': 'mem_wstrb'}, resp={})
bus_type = 'simple_mem'
irq = 'uart_irq'
name = 'uart0'
> parameters = {'registers': {'reg_div': {...}, 'reg_dat': {...}}, 'baudrate': 115200}
```

Intermediate Representation (IR)



STAGE 3 OUTPUT

```
✓ peripherals = [<ir.models.PeripheralIR object at 0x000001D269C6C770>, <ir.models.PeripheralIR object at 0x000001D269C6E3F0>, <ir.models.PeripheralIR object at 0x000001D269C6F050>]
✓ 0 = <ir.models.PeripheralIR object at 0x000001D269C6C770>
✓ 1 = <ir.models.PeripheralIR object at 0x000001D269C6E3F0>
  > special variables
  > bus_bindings = {'simple_mem': PeripheralBusBindingIR(bus_type='simple_mem', adapter='simple_mem_adapter', roles=Bus...m_wdata', 'wstrb': 'mem_wstrb'}, resp={})))
    bus_type = 'simple_mem'
    irq = 'uart_irq'
    name = 'uart0'
  > parameters = {'registers': {'reg_div': {...}, 'reg_dat': {...}}, 'baudrate': 115200}
  > region = AddressRegionIR(base_addr=33554436, size=4)
  > top_ports = {'ser_tx': {'dir': 'output'}, 'ser_rx': {'dir': 'input'}}
    type = 'simpleuart'
✓ 2 = <ir.models.PeripheralIR object at 0x000001D269C6F050>
  > special variables
  > bus_bindings = {'simple_mem': PeripheralBusBindingIR(bus_type='simple_mem', adapter='simple_mem_adapter', roles=Bus...ady': 'fft_ready', 'rdata': 'fft_rdata'}}))
    bus_type = 'simple_mem'
    irq = 'fft_irq'
    name = 'fft0'
  > parameters = {}
  > region = AddressRegionIR(base_addr=50331648, size=512)
  > special variables
    base_addr = 50331648
    size = 512
  > top_ports = {}
    type = 'fft_accel'
    len() = 3
  > reset = ResetIR(name='resetn', active_low=True)
```

Intermediate Representation (IR)



STAGE 3 OUTPUT

```
✓ IR saved to IR_output/

===== MEMORY MAP =====

Name           Type           Base Address   End Address    Size (Bytes)
-----
sram            memory          0x00000000     0x000003FF     1024
spimemio        peripheral       0x02000000     0x02000003      4
uart0           peripheral       0x02000004     0x02000007      4
fft0            peripheral       0x03000000     0x030001FF     512

=====

✓ File Generation is succesful and IR output are stored at IR_output as both JSON and YAML file
✓ Memory map saved to IR_output/memory_map.json

===== SoC Flow Completed Successfully upto IR Representation=====

hkremote@FFI-HK:~/Desktop/Final_Year_Project/VLSI_PROJECT-main/Automation$
```



memory_map.json



soc_ir_dump.json



soc_ir_dump.yaml

Memory Map and the IR stage output stored as JSON and YAML

Novelty & Innovation

- Automates RTL integration from a user-defined YAML configuration, eliminating manual RTL integration.
- Generates top-level interconnects and validates signal compatibility automatically for faster and error-free design.
- Enables rapid prototyping and architectural exploration through configuration-driven SoC customization.
- The framework is built to utilise custom built IPs , enabling low-cost, transparent top-level RTL generation.



Tools, Technologies, and Components

CORE & PERIPHERALS

- PicoRV32 Processor Core
- SRAM Module
- SPI Flash Memory
- GPIO
- Timer
- UART Interface
- 64-Point FFT Accelerator
- Interconnect Logic (Bus Architecture)

TOOLS & TECHNOLOGY USED

- Icarus Verilog
- GTKWave
- Python
- YAML
- VS Code



Feasibility

- **Technical Feasibility:** The reference SoC integrating PicoRV32, FFT accelerator, SRAM, UART, SPI Flash, GPIO, and Timer can be fully realized using open-source tools. Functional verification through Icarus Verilog and GTKWave.
- **Cost & Availability:** The entire toolchain is open-source, eliminating licensing and infrastructure costs. All required IP cores, and EDA tools, are freely available, enabling prototyping without reliance on proprietary software or costly hardware.
- **Team Capability:** The modular design approach allows clear task distribution among team members for RTL development and verification. The use of standardized open-source workflows ensures that the team can complete the project flow efficiently within the timeline.



Expected Outcomes

Stage-wise Outcomes

1) Specification Parsing

- Structured extraction of modules, parameters, and interfaces
- Conversion of parameters into dictionary

2) Specification Validation

- Detection of format/syntax errors, missing parameters, IQ and address overlapping.
- Ensures completeness and consistency of design inputs

3) IR Construction

- Generation of fabric-independent internal design model
- Unified representation of components and attributes

4) Address Allocation

- Assignment of base address from YAML input.

5) Connectivity Analysis

- Builds connectivity with the signals and Custom Bus (Memory

6) RTL Emission

- Generation of synthesizable top-level RTL
- Fabric-aware integration of modules and interconnect logic



Phase-II Execution Plan

January:

Front-end processing (Parsing & Validation)

February:

Internal modeling & mapping
(IR & Base Address Binding)

March:

System integration
(Connectivity & RTL Generation)

April:

Testing, debugging & final optimization

Applications

- **Rapid SoC Prototyping:**

Enables quick generation of custom SoC architectures directly from configuration files, significantly reducing manual design time.

- **Hardware Design Automation:**

Facilitates automation of top-level RTL, minimizing human error and streamlining integration of open-source IPs for academic or industrial ASIC design.

- **Domain-Specific SoCs:**

Supports creation of application-specific SoCs top-level RTL(e.g., for signal processing, IoT, or embedded AI) by modifying configuration parameters instead of rewriting RTL.



NAME	PROJECT CONTRIBUTION IN PHASE 1	PROJECT CONTRIBUTION IN Phase 2
Balasaraswathy B	Firmware Development and Testing, PicoSoC and FFT Accelerator Integration and Documentation.	Automation Tool Flow Design and development, Firmware Development and Testing, IP design and SoC Integration and Documentation.
Haresh Krishna G S	RDP Setup, Synthesis, RTL to GDSII cycle of PicoSoC and relevant reports generation and documentation.	High Level Architecture Design, Automation Tool Flow Design and Development, SoC Integration and Synthesis and Netlist creation using Yosys for Top-RTL wrapper and Documentation.
Harinee V T	Documentation.	Research on Timer IP & Documentation.

References

- A. P. D. Nath, K. Raj, S. Bhunia, and S. Ray, “**SOCCOM: Automated Synthesis of System-on-Chip Architectures**,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 40, no. 12, pp. 2451–2464, Dec. 2021.
- Defacto Technologies, “**Front-End SoC Integration Using SoC Compiler**,” *Defacto Technologies White Paper*, 2022.
- S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, “**RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique**,” *arXiv preprint arXiv:2312.08617*, Dec. 2023.
- T. S. R. Arachchige, “**CV-X-IF Based RISC-V Accelerator Generation**,” *Master’s Thesis, Tampere University*, Dec. 2024.
- Ithy Research Group, “**Revolutionizing SoC Design: Automated IP Component Integration in RTL**,” *Technical Report*, 2024.

